Guion de grabación — versión de una ojeada

EFT Desarrollo Backend III (PBY2203) · Ignacio Miño Astorga
Seis minutos · nueve tramos · el guion completo está en `guion_video.md`

> No leer esto en cámara. Son puntos de apoyo: los números van exactos, el resto con tus palabras.

Antes de empezar

Dejar abierto, cada cosa en su pestaña:

1. El repositorio en GitHub, carpeta `EFT_S9_Backend_Avanzado`
2. Una terminal con el sistema **ya levantado** (`docker compose ps` en healthy)
3. `informe/diagramas/01_arquitectura.png`
4. `evidencia/local/02_batch_migracion.log` · `09_bff_por_canal.log` · `08_tolerancia_a_fallos.log`
5. `evidencia/nube/05_metadatos_de_la_instancia.png` y `04_eureka_en_ec2.png`
6. Una terminal limpia con el bloque de los 200/403 ya copiado

---

0:00 – 0:40 · Presentación

Pantalla: webcam completa

- Ignacio Miño Astorga · Analista Programador Computacional · Duoc UC
- Banco XYZ: **30 años** sobre COBOL y Shell en mainframe
- El sistema funciona: lo justo para que cambiarlo parezca caro, lo suficiente para que cada cambio tome meses
- Lo que construí: **quince módulos**, tres frentes — batch en Spring Batch, un BFF por canal, microservicios con Spring Cloud
- Se levanta con un comando y escala horizontalmente

0:40 – 1:40 · Arquitectura

Pantalla: `01_arquitectura.png`, webcam en esquina, señalar con el cursor

- Cuatro capas, **una sola puerta de entrada**
- *(arriba)* Tres canales, tres BFF: cada uno agrega lo suyo y recorta el resto
- *(gateway)* Valida el token antes de enrutar; resuelve por **nombre de servicio en Eureka**, no por host y puerto → replicar sale gratis
- *(dominio)* Cuentas, pagos y clientes: cada uno dueño de sus datos y sus reglas
- *(mensajería)* Dos brokers a propósito — por qué, en un minuto

- *(batch)* Separado de todo: no es un servicio, se ejecuta y termina

---

1:40 – 2:30 · Batch y canales

Pantalla: `02_batch_migracion.log`, en el bloque del `FIN job`

- **1.000 leídas · 604 escritas · 396 omitidas**
- Las 396 no son un fallo: son los errores sembrados en el dataset —montos negativos, fechas imposibles, tipos inexistentes— y **cada una quedó registrada con su motivo**
- El proceso no se detuvo
- Corrió en **cuatro particiones en paralelo**, no en serie

Cambiar a: `09_bff_por_canal.log`, bloque de tamaños

- Misma cuenta, tres canales: **web 4.914 bytes · móvil 215 · cajero 39**
- Dos órdenes de magnitud para el mismo dato de negocio
- El cajero no necesita el historial de nadie: pedirlo sería gastar red y exponer lo que esa pantalla no muestra

---

2:30 – 3:30 · Seguridad y resiliencia

Pantalla: terminal. Pegar y ejecutar:

```
for recurso in /api/cuentas/101 /api/movimientos/101 /api/clientes/101; do
  for par in "web:$TOKEN_WEB" "movil:$TOKEN_MOVIL" "cajero:$TOKEN_CAJERO"; do
    canal="${par%%:*}"; tk="${par#*:}"
    printf "%-22s %-8s %s\n" "$recurso" "$canal" \
      "$(curl -s -o /dev/null -w '%{http_code}' -H "Authorization: Bearer $tk" "localhost:8080$recurso")"
  done
done
```

- Cada canal es un cliente OAuth 2.0 distinto, con sus propios scopes
- Web y móvil acceden a los tres recursos; **el cajero recibe 403** en movimientos y clientes, con un token de firma válida
- El permiso **no lo decide el gateway, lo decide cada microservicio** mirando el scope. Si estuviera solo en el gateway, cualquier proceso dentro de la red se lo saltaría

Cambiar a: `08_tolerancia_a_fallos.log`

- Detuve cuentas-service y pedí la ficha seis veces
- La primera responde degradada, circuito cerrado. **A la segunda el circuito se abre** y las siguientes responden de inmediato sin intentar la llamada

- Qué devuelve degradado: **el historial completo, los datos de cuenta en nulo**. Para quien consulta movimientos, el nombre del titular es un adorno
- *(bajar a las transiciones)* Ciclo completo: **cerrado → abierto → medio abierto → cerrado**, solo, sin que nadie tocara nada

---

3:30 – 4:00 · El despliegue en la nube

Pantalla: `05_metadatos_de_la_instancia.png`

- Todo esto también corre fuera de mi equipo: **una t3.large en us-east-1**
- Once imágenes construidas dentro de la instancia, doce servicios en healthy
- *(señalar)* Metadatos de la instancia leídos del servicio de metadatos de AWS, que **solo responde desde dentro de una máquina de Amazon**
- Sin esa cabecera, un `docker compose ps` es idéntico en cualquier computador y no probaría nada

Cambiar a: `04_eureka_en_ec2.png`

- La barra de direcciones: **una IP pública de Amazon**, no localhost
- `num-of-cpus` dice **2** — las dos vCPU de la instancia. En mi equipo dice **16**
- Lo que más quería demostrar: **Artemis y Kafka no están en mi portátil**. Los dos brokers corren en la nube y los microservicios se conectan desde la misma red privada

---

4:00 – 4:30 · Contra el sistema legacy

Pantalla: webcam, o la tabla comparativa del informe

- Batch: de secuencial a particionado, y de "se cae o no se cae" a **tres niveles de respuesta al fallo** — omitir y registrar, reintentar, relanzar el job
- Canales: de compartir un backend a **uno cada uno, con permisos distintos**
- Una caída: de afectar al sistema completo a **abrir un circuito y degradar**
- Escalar: de proyecto a bandera — `--scale cuentas-service=2`
- **Dónde el legacy es mejor hoy:** persiste sus datos y el mío no. Estado en memoria. Es el primero de mi lista

---

4:30 – 5:20 · Desafíos

Pantalla: webcam. Mirar a cámara, no leer.

Primero, de seguridad. Los dos aparecieron igual: auditando cuando ya creía terminado.

- Tenía OAuth 2.0 bien hecho, con scopes por canal y doble validación

- Y el Config Server servía la configuración completa **a cualquiera, sin credenciales** — y ahí viaja el secreto OAuth de un servicio
- Un GET anónimo entregaba una credencial para pedir un token válido: **todo el esquema de permisos neutralizado por la puerta de al lado**. Igual Eureka y el broker
- Lo arreglé con autenticación en los tres. Lo que me llevo no es la solución: **no lo encontré probando lo construido, sino preguntándome qué faltaba**
- La misma auditoría halló dos cosas que pasaron por delante de 136 pruebas en verde: un parser de fechas que mi documentación llamaba estricto y no lo era, y un DTO duplicado que mostraba **cero movimientos sobre un historial de cuarenta**. La suite quedó en **168**

Segundo, de consistencia.

- Una transferencia son dos llamadas remotas y no hay transacción que cubra a las dos
- Abono primero y falla el cargo → **creé dinero**. Cargo primero y falla el abono → descontré dinero que no llegó
- La segunda es reparable, la primera no: **cargar, abonar, y si el abono falla, devolver**. Si la compensación también falla, alerta de severidad alta con todo lo necesario para reponerlo a mano
- Lo incómodo: tuve que **escribir en el código que la solución está incompleta** — falta idempotencia y con estado en memoria no se sostiene—. Me costó, y creo que es lo más honesto del proyecto

---

5:20 – 5:50 · Qué falta

Pantalla: sección 9 del informe, o webcam

- **Persistencia**, y por una razón que no es obvia: demuestro escalado con dos instancias repartiéndose 15 y 15, y eso es real. Pero cada una tiene su copia del saldo en memoria: **apenas alguien retira, divergen**. El reparto funciona; la corrección del sistema escalado, no. Una demostración que funciona sin probar lo que parece probar
- **La clave de firma del auth-server fuera del proceso**: hoy se genera al arrancar, dos instancias firmarían distinto. Por eso corre con una sola y es el único punto único de falla
- Y tres más: **idempotencia**, **TLS con autenticación del usuario final**, y **trazabilidad distribuida** — hoy seguir un retiro son cinco logs cruzados a mano

---

5:50 – 6:00 · Cierre

Pantalla: webcam

- El código, la documentación completa y toda la evidencia están en el repositorio

- Gracias

Si algo se sale de tiempo

- **Largo:** recortar el tramo 0:40 – 1:40, dejando las cuatro capas y que el diagrama hable
- **Corto:** en 2:30, mostrar en vivo un retiro que baja el saldo en un servicio y aparece en el historial de otro por la cola
- **Nunca recortar el tramo 3:30:** es el único que demuestra algo que no se infiere del resto